Informe Proyecto 1 POO
Felipe Orengo y Lucas Ramírez

Clase abstracta: Pregunta

```java
 *
 * @author Lucas Ramírez y Felipe Orengo
 */
import java.util.Scanner;

public abstract class Pregunta {
    protected int peso;
    protected String text;

    public Pregunta(int peso, String text){
        this.peso = peso;
        this.text = text;
    }
    public int getPeso(){
        return peso;
    }

    public void setPeso(int peso){
        this.peso = peso;
    }

    public String getText(){
        return text;
    }

    public abstract boolean buscar(Scanner teclado);

}
```

Esta clase se hizo como una clase abstracta porque sirve de base o molde para todos los tipos de preguntas del examen. Aquí se declararon dos atributos protegidos “(protected)” para que las clases hijas los puedan usar directamente: ”peso”, que guarda los puntos que vale la pregunta, y ”text”, que guarda el enunciado como tal. Además de tener su constructor y sus métodos “getters” y ”setters” básicos para obtener o cambiar esos datos, lo más importante que tiene es el método abstracto ”buscar(Scanner teclado)”. Este método se deja vacío a propósito para obligar a que cada clase hija programe su propia forma de mostrar la pregunta y revisar si la respuesta está buena o mala.

Subclases de la clase abstracta:
Clase: TFpregunta

```java
/**
 * @author Lucas Ramírez y Felipe Orengo
 */
import java.util.Scanner;

public class TFpregunta extends Pregunta {
    private boolean respuestaCorrecta;

    public TFpregunta(String text, boolean respuestaCorrecta, int peso) {
        super(peso, text);
        this.respuestaCorrecta = respuestaCorrecta;
    }

    @Override
    public boolean buscar(Scanner teclado){
        System.out.print(text + "Ingrese T o F(verdadero o falso): ");
        while(true){
            String respuesta = teclado.nextLine().toUpperCase();

            if(respuesta.equals("T") || respuesta.equals("F")){
                boolean esVerdadero = respuesta.equals("T");
                if (esVerdadero == respuestaCorrecta){
                    System.out.println("Respuesta correcta");
                    return true;
                }else{
                    System.out.println("Respuesta incorrecta, la respuesta es: " + respuestaCorrecta);
                    return false;
                }
            }else{
                System.out.println("Entrada invalida, ingresa T o F: ");
                System.out.print(text + "Ingrese T o F(verdadero o falso): ");
            }

        }

    }


}
```

Esta clase hereda de “Pregunta” usando ”extends” y se encarga de las preguntas de verdadero o falso, por lo que se le agregó un atributo ”booleano” privado llamado ”respuestaCorrecta”. En su constructor recibe el texto, si es verdadero o falso, y el peso, pasándole el puntaje y el enunciado a la clase padre con ”super(peso, text)”. En el método ”buscar”, el programa muestra la pregunta y usa un ciclo ”while(true)” para leer lo que escribe el usuario pasándolo a mayúsculas con ”toUpperCase()”, así da lo mismo si lo escribe en minúscula. Si el usuario escribe otra cosa que no sea "T" o "F", el ciclo le avisa que la entrada es inválida y se lo vuelve a pedir; pero si escribe una opción válida, compara si acertó con ”respuestaCorrecta”, devolviendo ”true” si está bien o ”false” (mostrando cuál era la correcta) si se equivocó.

Clase: Resp_Cortas_Pregunta

```java
/**
 * @author Lucas Ramírez y Felipe Orengo
 */
import java.util.Scanner;

public class Resp_Cortas_Pregunta extends Pregunta{
    private String respuestaCorrecta;

    public Resp_Cortas_Pregunta(String text, String respuestaCorrecta, int peso){
        super(peso, text);
        this.respuestaCorrecta = respuestaCorrecta;
    }
    @Override
     public boolean buscar(Scanner teclado){
         System.out.print(text + ": ");
         String respuesta = teclado.nextLine();

         if(respuesta.equalsIgnoreCase(respuestaCorrecta)){
             System.out.println("Correcta");
             return true;
         }else{
             System.out.println("equivoado, la respuesta correcta es: " + respuestaCorrecta);
             return false;
         }
     }
}
```

Al igual que la anterior, esta clase hereda de “Pregunta”, pero está pensada para preguntas abiertas donde hay que escribir una respuesta corta, guardando la solución en un atributo ”String” llamado ”respuestaCorrecta”. Su constructor también usa ”super(peso, text)” para cargar los datos de la clase padre y guarda el texto de la respuesta correcta en su propio atributo. Cuando se ejecuta su método ”buscar”, simplemente imprime el enunciado en la consola y guarda todo lo que el usuario escriba con ”teclado.nextLine()”. Para revisar si está bien, ocupa el método ”equalsIgnoreCase()”, lo que permite comparar la respuesta del usuario con la correcta sin que importe si puso mayúsculas o minúsculas, retornando ”true” si coincide o ”false” mostrando la respuesta correcta si falla.

Clase: Selec_Mul_Pregunta

```java
 * @author Lucas Ramírez y Felipe Orengo
 */
import java.util.Scanner;

public class Selec_Mul_Pregunta extends Pregunta{
    private String[] opciones;
    private int indexCorrecto;

    public Selec_Mul_Pregunta(String text, String[] opciones, int indexCorrecto, int peso) {
        super(peso, text);
        this.opciones = opciones;
        this.indexCorrecto = indexCorrecto;
    }

    @Override
    public boolean buscar(Scanner teclado){
        System.out.println(text);
        char letra = 'a';
        for (int i = 0; i < opciones.length; i++) {
            System.out.println("  " + (char)(letra + i) + ". " + opciones[i]);
        }

        while(true){
        System.out.print("Ingrese la letra de su eleccion: ");
        String respuesta = teclado.nextLine().toLowerCase();

        if (respuesta.length() > 0) {
    int opcionElegida = respuesta.charAt(0) - 'a';

        if (opcionElegida >= 0 && opcionElegida < opciones.length) {

        if (opcionElegida == indexCorrecto) {
            System.out.println("Correcto");
            return true;
        } else {
            char letraCorrecta = (char) ('a' + indexCorrecto);
            System.out.println("Incorrecto, la respuesta es: " + letraCorrecta);
            return false;
        }
      } else {
    System.out.println("Error. Por favor, elige una de las opciones del menu.");
     }
        } else {
                System.out.println("No escribiste nada. Intenta de nuevo.");
      }
    }
  }
```

Esta clase también extiende de “Pregunta” y sirve para hacer preguntas con alternativas, por lo que guarda un arreglo de textos ”(String[] opciones)” con las opciones disponibles y un entero ”(indexCorrecto)” que indica en qué posición del arreglo está la respuesta buena. En su método ”buscar”, primero muestra la pregunta y después recorre el arreglo con un ciclo ”for”, sumándole la posición ”i” a la letra ”'a” para que las alternativas se impriman solas con letras en orden (a., b., c., etc.). Luego entra a un ciclo ”while(true)” que pasa la letra elegida a minúscula y le resta el carácter ”a” ”(respuesta.charAt(0) - ”a”)” para transformarla en un número y así poder compararla con las posiciones del arreglo. Si el usuario deja el espacio en blanco o pone una letra que no está en el menú, le pide que intente de nuevo, y si elige una letra válida, revisa si calza con ”indexCorrecto” para retornar ”true” o ”false” diciendo cuál era la letra correcta.

Clase: Exam

```java
 * @author Lucas Ramírez y Felipe Orengo
 */
import java.util.ArrayList;
import java.util.Scanner;

public class Exam {
    private ArrayList<Pregunta> preguntas;

    public Exam(){
        preguntas = new ArrayList<>();
    }

    public void agregaPregunta(Pregunta p){
        if (preguntas.size() < 10){
            preguntas.add(p);
        }else{
            System.out.println("El examen alcanzo el límite de preguntas.");
        }
    }
    public int darExam(){
        Scanner teclado = new Scanner(System.in);
        int puntajeObtenido = 0;
        int puntajeTotal = 0;

        for (Pregunta p : preguntas){
            puntajeTotal += p.getPeso();

            boolean acierto = p.buscar(teclado);

            if(acierto){
                puntajeObtenido += p.getPeso();
            }
        }
        System.out.println("--------------------------------");
        System.out.println("RESULTADOS DEL EXAMEN");
        System.out.println("Puntaje: " + puntajeObtenido + "/" + puntajeTotal);

    if (puntajeTotal == 0) return 0;
    return (int) Math.round(((double) puntajeObtenido / puntajeTotal) * 100);

    }

}
```

Esta clase es la que administra todo el examen y para eso usa un “ArrayList<Pregunta>” llamado ”preguntas”, donde gracias al polimorfismo se pueden guardar mezcladas las preguntas de verdadero/falso, respuesta corta y selección múltiple en una misma lista. Tiene el método ”agregaPregunta()”, que revisa con un ”if” que la lista tenga menos de 10 preguntas antes de agregar una nueva, tirando un mensaje de aviso si ya se llegó al tope. Aparte, tiene el método ”darExam()”, que recorre toda la lista con un ciclo ”for”, va sumando el puntaje total posible y llama al método ”buscar” de cada pregunta para que el usuario la responda en el momento. Si el usuario acierta, se le suma el peso de esa pregunta a su puntaje obtenido, y al terminar el ciclo se muestra el puntaje final junto con el cálculo del porcentaje redondeado usando ”Math.round()”.

Clase main: ExamDemo

```java
/**
 * @author Lucas Ramírez y Felipe Orengo
 */
public class ExamDemo {
    public static void main(String[] args){
        Exam miExam = new Exam();

        System.out.println("_____*EXAMEN PROYECTO 1 POO*_____");

        //preguntas TF
        miExam.agregaPregunta(new TFpregunta("El perro que visita la UCEN se llama Trigo? ", true, 1));

        miExam.agregaPregunta(new TFpregunta("Visual Studio Code es un lenguaje de programacion? ", false, 2));

        miExam.agregaPregunta(new TFpregunta("Yoscar Daza es Erling Haaland? ", true, 1));

        //preguntas respcortas
        miExam.agregaPregunta(new Resp_Cortas_Pregunta("Que campeon de lol es el favorito del Yoscar? ", "DR mundo", 1));

        miExam.agregaPregunta(new Resp_Cortas_Pregunta("Cual es el apodo de Fabian Perez? ", "Shadow", 1));

        miExam.agregaPregunta(new Resp_Cortas_Pregunta("Cuales son las siglas de Programacion Orientada a Objetos? ", "POO", 1));

        //preguntas selecMul
        String[] opcionesCarrera = {"Jonatan Vignatti", "Gonzalo Honores", "Fabian Perez", "Yoscar Daza"};
        miExam.agregaPregunta(new Selec_Mul_Pregunta( "Quien es el director de carrera de ICCI?: ", opcionesCarrera, 1, 1));

        String[] opcionIA = {"IOscar Daza", "Ingenieria computacional", "Inteligencia Artificial", "Computadores", "fabIAn Perez"};
        miExam.agregaPregunta(new Selec_Mul_Pregunta("Que es IA?", opcionIA, 2, 1));

        String[] opcionHardware = {"Un grupo musical", "Una aplicacion", "Un algoritmo", "Recursos de un pc"};
        miExam.agregaPregunta(new Selec_Mul_Pregunta("Que es hardware?", opcionHardware, 3, 1));

        int puntaje = miExam.darExam();

        System.out.println("Su porcentaje es: " + puntaje + "%");
    }
}
```

Por último, esta es la clase principal que tiene el “main” para hacer correr el programa y probar que todo funcione bien. Lo que hace es crear el objeto ”miExam” y cargarle 9 preguntas de prueba usando ”agregaPregunta()”: se pusieron 3 de verdadero o falso, 3 de respuesta corta y 3 de selección múltiple (creando antes los arreglos ”String[]” con las alternativas de cada una y dándole distintos pesos e índices según el caso). Una vez que están todas las preguntas listas, se llama a ”miExam.darExam()” para empezar a responder el cuestionario en la consola y, cuando termina, guarda el resultado en la variable ”puntaje” para imprimir en pantalla el porcentaje final que sacó el usuario.

Diagrama UML hecho con Lucidchart:

